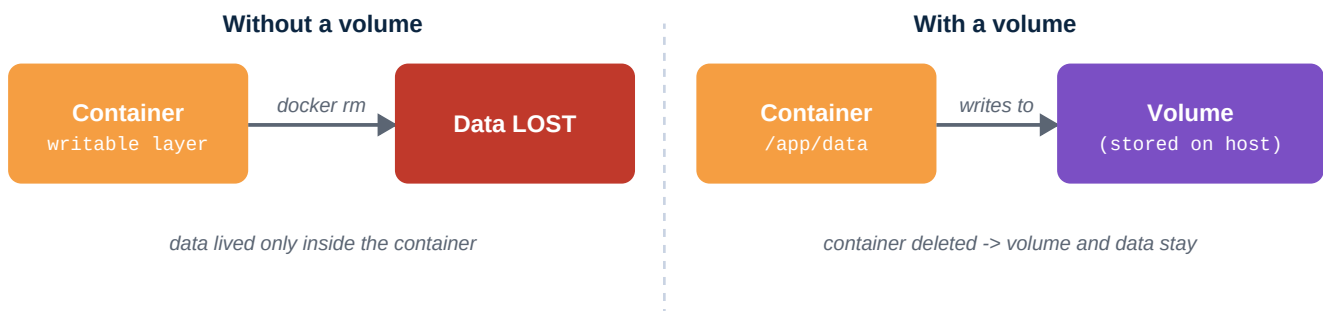


Docker Notes: Set 5

Persisting data with volumes and bind mounts, Docker Compose, and deploying a website on Nginx

1. Persisting Data in Docker

A container's writable layer is temporary: **if the container is deleted, its data is lost**. To keep data you store it outside the container.



Why persist data?

- **Persist data** (for example database files)
- **Share data** between containers
- **Separate app code from user data**
- **Avoid losing data** when a container stops or is deleted

The two common ways: **Volumes** and **Bind mounts**. Both use the `-v` option of `docker run`.

	Volume	Bind mount
Stored where	Docker-managed area: <code>/var/lib/docker/volumes/NAME/_data</code>	Anywhere on the host file system
Syntax	<code>-v mydata:/app/data</code>	<code>-v /host/path:/app</code>
Managed by	Docker (docker volume commands)	You (normal files and folders)
Best for	Databases, production data, sharing between containers	Development (live code), config files
Where set	At run time (docker run or Compose), not in the Dockerfile	Same

Tip: Quick rule: if the left side of `-v` is a plain **name** (mydata) it is a volume; if it is a **path** starting with `/` or `$` (pwd) it is a bind mount.

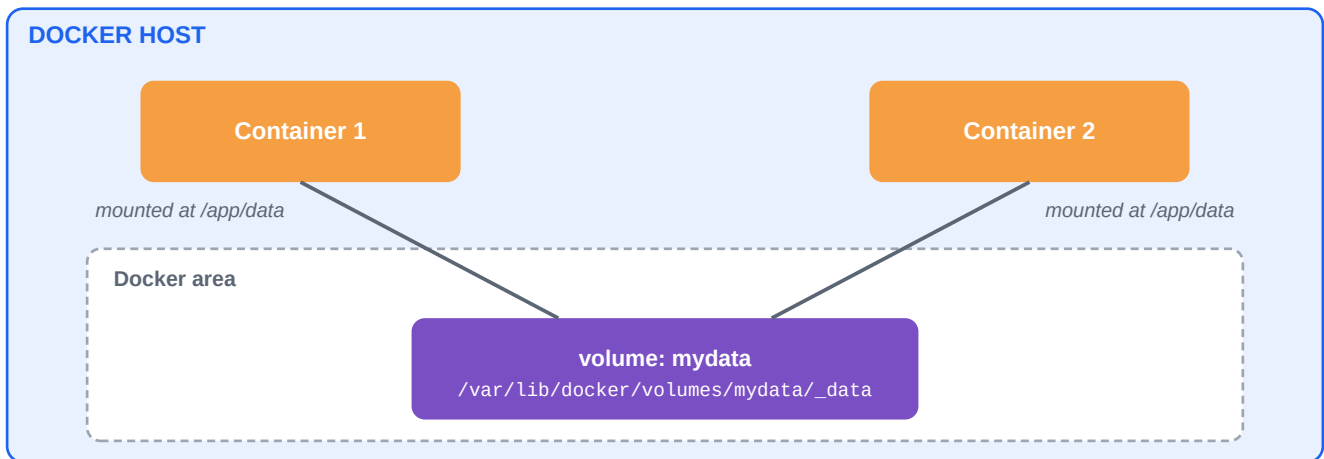
2. Data Volumes

Volumes are stored on the host file system, inside Docker's own storage area, and managed by Docker. Each volume gets its own path:

```
/var/lib/docker/volumes/<volume-name>/_data/
```

- **Isolated** from the container layers
- **Persists independently** of containers

- **Mounted into containers** at run time
- Data survives container restarts, deletion and image rebuilds



Volume commands

Command	Purpose
<code>docker volume create NAME</code>	Create a new named volume
<code>docker volume ls</code>	List existing volumes
<code>docker volume inspect NAME</code>	Show detailed info (mountpoint, driver, creation date)
<code>docker volume rm NAME</code>	Delete a volume (it must not be in use by a container)

Using a volume with docker run

```
-v VOLUME_NAME:CONTAINER_DIR          # read-write (default)
-v VOLUME_NAME:CONTAINER_DIR:ro       # read-only for the container
```

```
# creates volume mydata (if missing) and mounts it at /app/data
docker run -v mydata:/app/data ubuntu
# same volume, but the container cannot write to it
docker run -v mydata:/app/data:ro ubuntu
```

Careful: In your notes the read-only example had a space: `mydata:/app/data :ro`. There must be **no space**: `mydata:/app/data:ro`.

Demo: data survives a deleted container

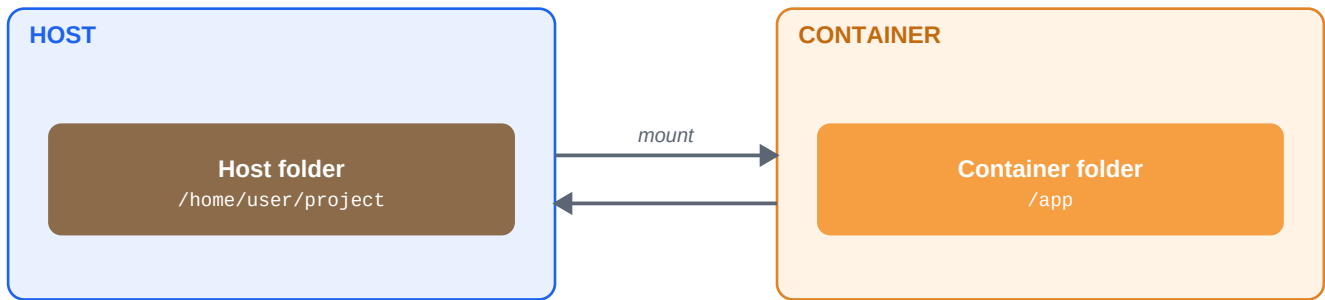
```
docker volume create mydata
docker run -it --name c1 -v mydata:/app/data ubuntu bash
root@c1:/# echo "hello" > /app/data/note.txt
root@c1:/# exit

docker rm c1                                     # container is gone...
docker run --rm -v mydata:/app/data ubuntu cat /app/data/note.txt
hello                                           # ...but the data is still there

docker volume inspect mydata                   # shows "Mountpoint":
#  "/var/lib/docker/volumes/mydata/_data"
```

3. Bind Mounts

A bind mount links a **host path** (file or folder) directly to a **container path**. The host path can be anywhere on the host. It is used with `docker run -v` (or Compose), **not** in the Dockerfile. Perfect for development and testing because the container can directly use your host files.



Edit on host = instantly visible in the container (and the other way round)

Command	What it does
<code>docker run -v /home/user/project:/app myimage</code>	Container folder /app is mapped to the host folder /home/user/project
<code>docker run -v \$(pwd):/app my-dev-image</code>	Mounts the current folder; every change on the host is immediately visible in the container
<code>docker run -v /home/user/nginx.conf:/etc/nginx/nginx.conf:ro nginx</code>	Injects a host-specific config file into the container without rebuilding the image (:ro = read-only)

Example: live-edit a website without rebuilding

```
mkdir site && echo "<h1>Hello v1</h1>" > site/index.html
docker run -d --name web -p 8080:80 -v $(pwd)/site:/usr/share/nginx/html:ro nginx

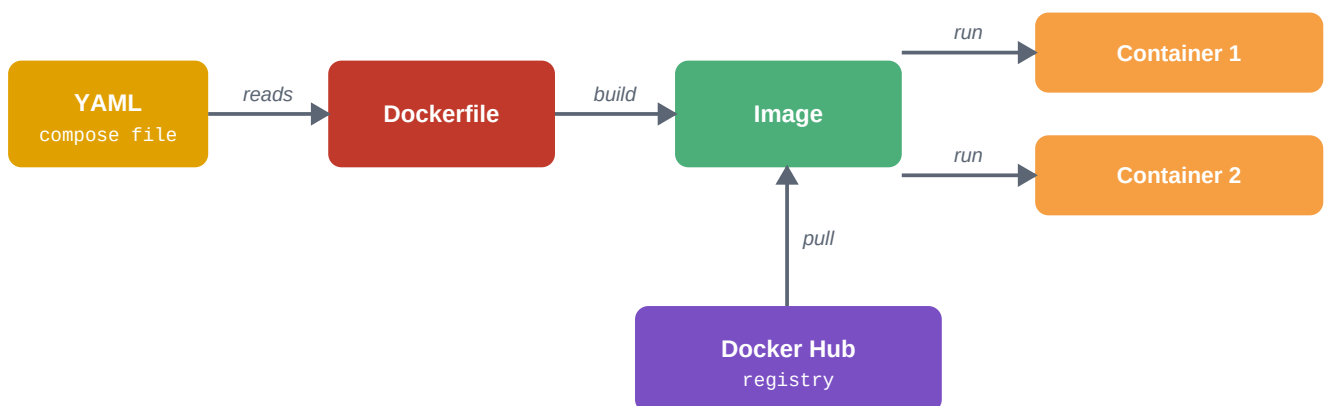
# open http://localhost:8080 -> Hello v1
echo "<h1>Hello v2</h1>" > site/index.html
# refresh the browser -> Hello v2 (no rebuild, no restart)
```

Note: Use an **absolute path** (or \$(pwd)) on the left side. A relative name like `site:/app` would be treated as a *volume* called `site`, not a folder.

4. Docker Compose

Docker Compose defines and runs **multi-container** applications. You describe the whole environment (services, networks, volumes, dependencies) in one YAML file, `docker-compose.yml`, and start everything with a single command. Compose can **build** images from Dockerfiles or **pull** them from a registry.

One YAML file describes every service. Compose builds from a Dockerfile (build:) or pulls a ready image (image:).

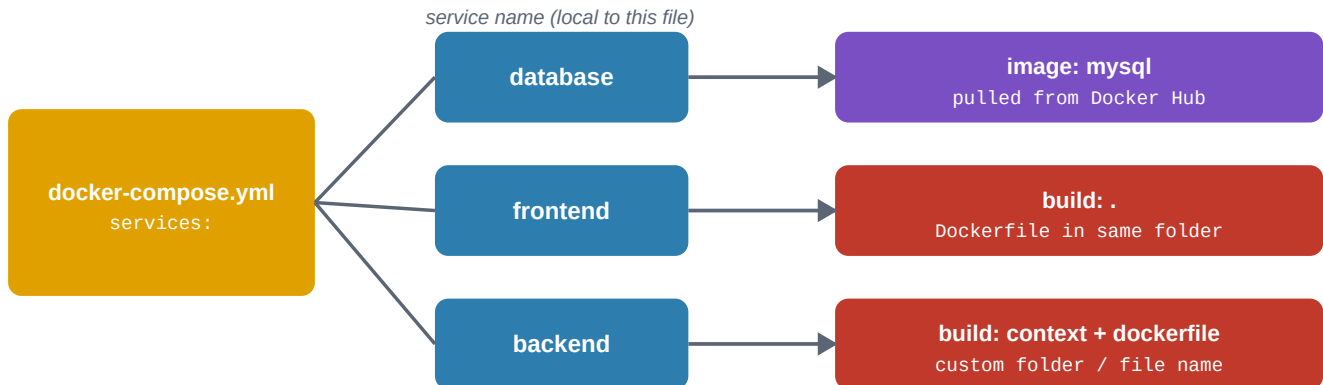


```
# build/pull images, create network + containers, start everything
docker compose up -d
# stop and remove them all
docker compose down
```

Note: Newer Docker uses `docker compose` (with a space). Older installs use `docker -compose` (with a hyphen). The commands work the same.

5. Compose File Structure

Under `services:` each entry is one container. In the example there are three services: **database**, **frontend** and **backend**. These names only have meaning inside the Compose file (but they also become host names on the network, see section 7).



Key	Meaning	Example
image	Use a ready image (pulled from Docker Hub)	image: mysql:8
build	Build from a Dockerfile. Short form = folder of the Dockerfile, next to the compose file	build: .
build: context / dockerfile	Full form: folder to build from, and the Dockerfile's file name	context: ./app dockerfile: fileconfig
container_name	Fixed name for the container	container_name: mydb
ports	HOST:CONTAINER port mapping (quote it)	- "3000:5000"
volumes	Mount a bind mount, named volume or anonymous volume (see below)	- db-data:/var/lib/mysql

The three kinds of volumes in Compose

Type	Written as	Notes
Bind mount	- hostPath:containerPath	Example: - ./html:/usr/share/nginx/html
Named volume	- volumeName:containerPath	Must also be declared in the top-level volumes: section
Anonymous volume	- containerPath	Docker generates a random name; no top-level volumes: section needed

A complete docker-compose.yml

```
services:
  database:
    image: mysql:8
    container_name: mydb
    environment:
      MYSQL_ROOT_PASSWORD: example
    volumes:
      - db-data:/var/lib/mysql      # named volume (declared below)

  backend:
    build:
      context: ./app                # folder containing the Dockerfile
      dockerfile: fileconfig        # custom Dockerfile name

  frontend:
    build: .                        # Dockerfile next to this file
    ports:
      - "3000:5000"                # host 3000 -> container 5000

volumes:
  db-data:                         # create the named volume
```

Note: Compose files use **spaces for indentation** (never tabs), and the indentation decides what belongs to what. The mysql image needs MYSQL_ROOT_PASSWORD to start. Comments in the YAML above are for explanation; YAML allows them (#).

6. Docker Compose Commands

Command	Purpose
<code>docker compose build</code>	Build the images for the services
<code>docker compose run SERVICE</code>	Create and run a container for one specific service from the file
<code>docker compose up</code>	Build images if they are not found locally, then create and start the containers
<code>docker compose -f FILE up</code>	Same, but use a different compose file
<code>docker compose up -d</code>	Same as up, but in detached (background) mode
<code>docker compose down</code>	Stop and remove all containers created by up (and the network)
<code>docker compose down --volumes</code>	Also remove the volumes (named and anonymous)
<code>docker compose down --remove-orphans</code>	Also remove old (orphan) containers no longer in the YAML file
<code>docker compose down --volumes --remove-orphans</code>	All of the above combined

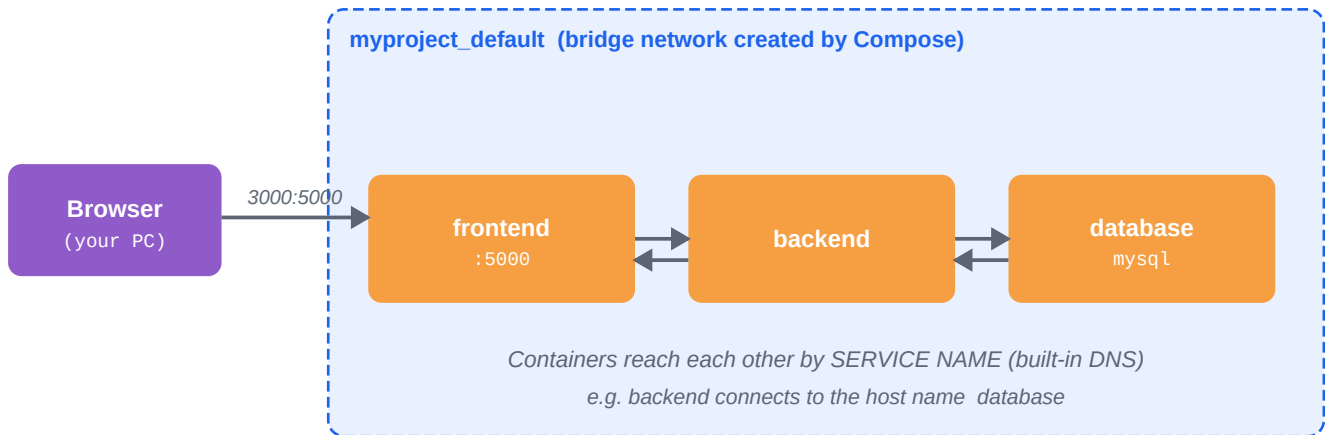
Careful: Your note wrote `-f` with an en-dash. Type a normal hyphen: `-f`. Also remember: plain down keeps your volumes, so database data survives. Only `--volumes` deletes it.

Everyday workflow

```
docker compose up -d --build      # start (rebuild images if code changed)
docker compose ps                 # list the project's containers
docker compose logs -f frontend  # follow logs of one service
docker compose down               # stop and clean up (data in volumes is kept)
```

7. Docker Compose Network

By default Compose creates a **bridge network** for your app. Every service joins it and is reachable by the other services using the **service name** as the host name. The network is named after the project folder plus `_default`.



```
docker compose up -d
docker network ls
```

NETWORK ID	NAME	DRIVER	SCOPE
880f38a30a3e	bridge	bridge	local
b7dd88f96729	host	host	local
d6e109ba1caa	ubuntu_default	bridge	local

<- created by Compose

Note: Only the ports : mapping (3000:5000) exposes a service to your browser. Between containers, no port mapping is needed: the backend simply connects to database:3306.

8. Deploy a Website on Nginx Using a Dockerfile

Goal: package a static website into an Nginx image. Nginx serves files from `/usr/share/nginx/html/`, so the Dockerfile copies the site there.

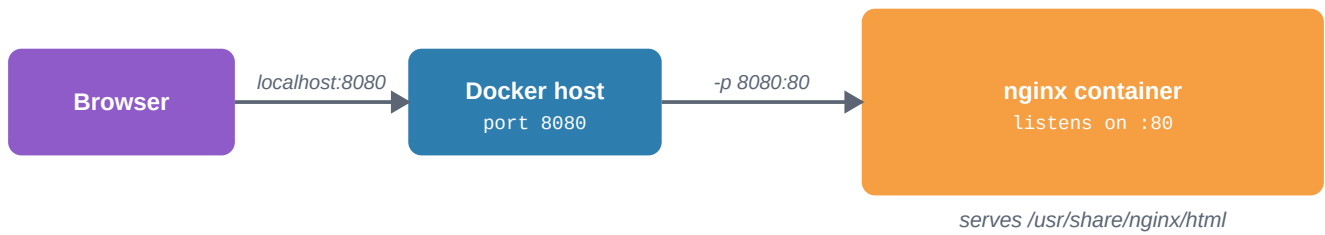
Docker host (project folder)	Inside the container image
<pre>/home/ubuntu/ -- Docker-Capstone-Project1/ -- Dockerfile -- docker-compose.yml (optional) -- school-website/ -- index.html -- error.html -- about.html -- assets/ -- Documentation/ -- src/</pre>	<pre>/usr/share/nginx/html/ -- index.html -- error.html -- about.html -- assets/ -- Documentation/ -- src/ (COPY places the contents of school-website/ here)</pre>

Dockerfile

```
FROM nginx:latest
COPY school-website/ /usr/share/nginx/html/
EXPOSE 80
```

The nginx image already starts the web server, so no CMD is needed. Build and run from the project folder:

```
cd /home/ubuntu/Docker-Capstone-Project1
docker build -t school-site .
docker run -d --name school -p 8080:80 school-site
# open http://localhost:8080 in the browser
```



Same site with Docker Compose (optional file in the tree)

```
services:
  web:
    build: .
    ports:
      - "8080:80"
```

```
docker compose up -d      # builds school-site image and starts it
docker compose down       # stop and remove
```

Tip: While developing, skip rebuilds with a bind mount: `docker run -d -p 8080:80 -v $(pwd)/school-website:/usr/share/nginx/html:ro nginx`. Use the Dockerfile version for the final, shareable image.

9. Quick Recap

- Container data is lost on delete; use a **volume** (Docker-managed) or **bind mount** (host path) to keep it.
- `-v name:/path` = volume, `-v /host/path:/path` = bind mount, add `:ro` (no space) for read-only.
- Volume commands: `create`, `ls`, `inspect`, `rm`.
- **Compose** = one YAML file for many containers: `services`, `image`, `build`, `container_name`, `ports`, `volumes`.
- `up -d` starts everything; `down` removes containers; `down --volumes` also removes data.
- Compose services share a default bridge network and find each other by **service name**.
- For a static site: `FROM nginx + COPY site/ /usr/share/nginx/html/ + -p 8080:80`.